Redux – Extension des fonctionnalités avec les Middlewares

Pourquoi utiliser des middlewares ?

Dans Redux, un middleware permet d'intercepter les actions **entre le dispatch et le reducer**.

Schéma :

dispatch(action) → MIDDLEWARE → reducer → state

Les middlewares permettent :

1. **Observer les actions** (ex : logger)
2. **Gérer les actions asynchrones** (ex : fetch API)
3. **Bloquer / modifier certaines actions**
4. **Faire des vérifications ou audits**
5. **Gérer l'authentification, permissions, logs…**

Sans middleware, Redux ne sait gérer que **des actions synchrones**.

Les middlewares les plus importants :

1. redux-logger

Middleware qui affiche dans la console :

- l'action envoyée
- l'état avant
- l'état après

Très utilisé en formation / débogage.

2. redux-thunk

Middleware qui permet d'envoyer **des fonctions** au lieu d'actions.

Permet de gérer :

- les appels API
- le async/await
- le chargement avant et après une requête

Formateur : AARAB HAFID

Bien que Redux Toolkit intègre déjà thunk par défaut, il est important de comprendre son rôle.

Comment ajouter un middleware dans Redux Toolkit ?

Fichier : store.js

```
import { configureStore } from "@reduxjs/toolkit";
import tasksReducer from "./tasksSlice";
// Import des middlewares
import logger from "redux-logger"; // middleware de logs

const store = configureStore({
  reducer: {
    tasks: tasksReducer
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(logger)
});

export default store;
```

Explications :

1. getDefaultMiddleware() → inclut thunk automatiquement.
2. .concat(logger) → ajoute redux-logger à la chaîne des middlewares.
3. Chaque action sera maintenant affichée dans la console.

Utilité des middlewares :

Concept	Explication
Logger	Pour afficher actions + state dans la console
Thunk	Pour gérer les fonctions async (fetch API)

Concept	Explication
Custom middleware	Pour sécuriser, bloquer, ou vérifier les actions

Utiliser redux-logger

Installation

npm install redux-logger

Effet visible

Quand on dispatch une action :

dispatch(addTask({id: 1, titre: "Cours Redux", completed:false}))

La console affiche :

prev state: { tasks: [] }

action: { type: "tasks/addTask", payload: {…} }

next state: { tasks: [{…}] }

Utiliser redux-thunk (déjà intégré dans Redux Toolkit)

Même si Redux Toolkit intègre thunk automatiquement, il faut expliquer **le concept**.

Sans thunk → impossible :

dispatch(async () => {})

Avec thunk → autorisé :

On peut dispatcher une fonction :

```
export const fetchTasks = () => {
  return async (dispatch) => {
    dispatch({ type: "tasks/loading" });

    const res = await fetch("https://jsonplaceholder.typicode.com/todos");
    const data = await res.json();

    dispatch({ type: "tasks/setTasks", payload: data });
```

```
  };
};
```

C’est thunk qui rend ça possible.

Exemple simple : Middleware personnalisé (login)

Créer un middleware pour surveiller les actions liées au login.

Fichier : middlewares/loginMiddleware.js

```
const loginMiddleware = store => next => action => {
  if (action.type === "user/login") {
    console.log("Tentative de connexion :", action.payload.username);
  }

  return next(action); // passer à l'action suivante
};

export default loginMiddleware;
```

le middleware peut lire / modifier / bloquer une action.

Ajouter le middleware dans Redux Toolkit

```
import loginMiddleware from "./middlewares/loginMiddleware";
const store = configureStore({
  reducer: {
    tasks: tasksReducer
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(loginMiddleware)
});
```

TP Final – Redux Toolkit

Créer une mini-application React + Redux Toolkit avec :

- un slice tasks
- un appel API via createAsyncThunk
- un middleware personnalisé (logs)
- redux-logger
- affichage état loading / erreur

Créer le slice

Dans tasksSlice.js :

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

// chargement API
export const fetchTasks = createAsyncThunk(
  "tasks/fetch",
  async () => {
    const res = await fetch("https://jsonplaceholder.typicode.com/todos?_limit=5");
    return res.json();
  }
);

const tasksSlice = createSlice({
  name: "tasks",
  initialState: { tasks: [], loading: false, error: null },
  reducers: {
    addTask: (state, action) => {
      state.tasks.push(action.payload);
    }
    },
```

```
  extraReducers: (builder) => {
    builder.addCase(fetchTasks.pending, (state) => {
      state.loading = true;
    });

    builder.addCase(fetchTasks.fulfilled, (state, action) => {
      state.loading = false;
      state.tasks = action.payload;
    });

    builder.addCase(fetchTasks.rejected, (state) => {
      state.loading = false;
      state.error = "Erreur API";
    });
  }
});

export const { addTask } = tasksSlice.actions;
export default tasksSlice.reducer;
```

2/Ajouter middlewares dans store.js

```
import { configureStore } from "@reduxjs/toolkit";
import tasksReducer from "./tasksSlice";
import logger from "redux-logger";

const alertMiddleware = store => next => action => {
  console.log("ACTION DISPATCHÉE:", action.type);
  return next(action);
```

```
};

const store = configureStore({
  reducer: { tasks: tasksReducer },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(logger, alertMiddleware)
});

export default store;
```

3/ Interface React simple (App.js)

```
import { useDispatch, useSelector } from "react-redux";
import { fetchTasks, addTask } from "./tasksSlice";

function App() {
  const dispatch = useDispatch();
  const { tasks, loading } = useSelector(state => state.tasks);

  return (
    <div>
      <h1>TP Final – Redux Toolkit + Middlewares</h1>

      <button onClick={() => dispatch(fetchTasks())}>
        Charger depuis API
      </button>

      <button onClick={() => dispatch(addTask({ id: Date.now(), title: "Nouvelle tâche" }))}>
        Ajouter tâche
```

```
    </button>

    {loading && <p>Chargement...</p>}

    <ul>
      {tasks.map(t => (
        <li key={t.id}>{t.title}</li>
      ))}
    </ul>
  </div>
 );
}

export default App;
```

Résumé

Concept	Utilité simple
Middleware	Intercepte les actions
redux-logger	Affiche actions + state pour débogage
redux-thunk	Permet async/await dans Redux
Middleware perso	Surveiller, bloquer ou modifier des actions
Redux Toolkit	configureStore + createSlice + thunk intégré